

Code protocol: the model writes the method

Ask for a solve() function
no tools, no examples

- 1 every given value → a defaulted parameter
- 2 return {"value", "unit"} per asked quantity
- 3 standard library only – never hard-code the answer

Write and run the code

```
def solve(A=0.30, F_s=1368):  
    T = ((1-A)*F_s/(4*5.67e-8))*0.25  
    return {"1": {"value": T, "unit": "K"}}  
-----  
executed in a sandbox – hard 10s timeout  
will not run → code + its error fed back  
still failing after 5 tries → FAIL
```

Numeric answers

```
{ "1": { "value": 254.9, "unit": "K" } }
```

* the interpreter did the arithmetic

Ask for the final number
no tools, no examples

- 1 use the given values + standard constants
- 2 one value per asked quantity, in order
- 3 report each answer as \boxed{number unit}

Reason and answer in prose

At equilibrium, $(1-A)F_s/4 = \sigma T^4$
so, $(0.70 \times 1368 / (4\sigma))^{0.25} \approx 254.9 \text{ K}$

the arithmetic happens in the text – nothing is executed
no \boxed{} value → reminder fed back
still boxless after 5 tries → FAIL

Numeric answers

\boxed{254.9 K}

* the model itself did the arithmetic

Direct protocol: the model computes in prose

Identical numeric grading

accuracy = $\frac{\text{passed}}{\text{passed} + \text{failed}}$

PASS
every asked quantity within tolerance

FAIL
any quantity outside tolerance

a wrong value is final – repair fixes format, never the physics

Compare with the frozen ground truth

254.9 K vs 255 K → 0.04%

each asked quantity: relative error ≤ 5%
any listed acceptable value counts
simple units reconciled first (K, km, hPa, %)

Benchmark problem
the model sees only this text

Calculate Earth's equivalent blackbody temperature at radiative equilibrium, given albedo $A = 0.30$ and solar flux $F_s = 1368 \text{ W m}^{-2}$.

no formulas · no hints ·
no expected answer